

Naive Bayes: A Worked Example in Code

Contents

1. Overview	2
2. The data	2
3. Two phases	2
4. Training: the lookup table	3
5. Testing: look up and multiply	3
6. The zero-frequency problem	4
7. The fix: Laplace smoothing	4
8. Summary	6
9. Sources	6
10. Key terms	6

Where this fits: Pipeline map step 8 (Model). See the [Course map](#).



- **Builds on:** Classification problems ([Note 3](#)); Probability density function (PDF) ([Note 20](#)); Normal distribution ([Note 42](#)); Independent and mutually exclusive events ([Note 84](#)); Bayes' theorem ([Note 86](#)).

1. Overview

Key point: Training Naive Bayes means building a lookup table of probabilities by counting. Predicting means looking up one probability per feature and multiplying.

The previous two Notes gave the intuition and the formula. This Note applies them to a classic toy dataset, **Play Tennis**, in Python: first by hand with pandas, then with scikit-learn. It also meets a practical problem, the **zero-frequency problem**, and its standard fix.

2. The data

Key point: 14 days, four weather features, and whether tennis was played (9 yes, 5 no).

Each day is one **observation** (one record, a row of the data table). Each has four **features** (input variables, one column each): **outlook** (sunny, overcast, rain), **temperature** (hot, mild, cool), **humidity** (high, normal) and **wind** (weak, strong). The **target** (the output we predict) is **play** (yes or no). Tennis was played on 9 of the 14 days.

The question: on a day that is sunny, hot, high-humidity with weak wind, will tennis be played?

3. Two phases

Key point: Training computes and stores every probability; testing only looks them up.

Like every machine learning algorithm, Naive Bayes works in two phases:

1. **Training:** go through the data once and compute every probability the formula could ever need: the class priors, and $P(\text{value} \mid \text{class})$ for every value of every feature. Store them in a **lookup table** (in Python, a dictionary).
2. **Testing:** for a new day, look up the relevant probabilities and multiply. Nothing is recomputed.

How many probabilities does the table hold? Outlook has 3 values and there are 2 classes, so 6; temperature 6; humidity 4; wind 4; plus 2 priors. 22 numbers in all.

4. Training: the lookup table

Key point: `pd.crosstab` counts each value per class; dividing by the class sizes gives $P(\text{value} \mid \text{class})$.

Python: Building the lookup table.

```
cols = ["outlook", "temperature", "humidity", "wind"]
class_counts = df["play"].value_counts()           # Yes 9, No 5
prior = class_counts / len(df)                     # 9/14, 5/14
lookup = {col: pd.crosstab(df[col], df["play"]) / class_counts
          for col in cols}
```

`pd.crosstab` counts how often each value appears with each class: for outlook, sunny appears on 3 “no” days and 2 “yes” days. Dividing each class column by the class size turns the counts into probabilities.

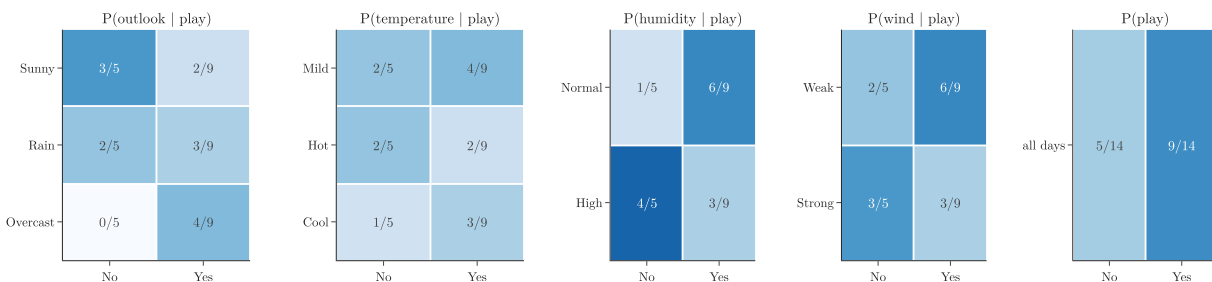


Figure 1: The full lookup table: $P(\text{value} \mid \text{play})$ for every feature, and $P(\text{play})$

For example, of the 5 days without tennis, 4 had high humidity, so $P(\text{high} \mid \text{no}) = 4/5$; of the 9 days with tennis, 6 had normal humidity, so $P(\text{normal} \mid \text{yes}) = 6/9$.

5. Testing: look up and multiply

Key point: Sunny, hot, high, weak: yes scores 0.0071, no scores 0.0274. Predict no.

Python: Prediction.

```
def predict(day):
    scores = {}
    for c in ["Yes", "No"]:
        s = prior[c]
        for col, value in day.items():
            s *= lookup[col].loc[value, c]
        scores[c] = s
    return scores
```

```
predict({"outlook": "Sunny", "temperature": "Hot",  
        "humidity": "High", "wind": "Weak"})
```

For the sunny, hot, high, weak day:

$$\text{yes: } \frac{9}{14} \times \frac{2}{9} \times \frac{2}{9} \times \frac{3}{9} \times \frac{6}{9} = 0.0071$$

$$\text{no: } \frac{5}{14} \times \frac{3}{5} \times \frac{2}{5} \times \frac{4}{5} \times \frac{2}{5} = 0.0274$$

The “no” score is larger: **no tennis**. As probabilities, $0.0274/(0.0071 + 0.0274) = 0.795$, so 79.5% no. Sunny weather and high humidity, both much more common on “no” days, decide it.

6. The zero-frequency problem

Key point: Overcast never occurred on a “no” day, so $P(\text{overcast} \mid \text{no}) = 0$, and any overcast day gets a “no” score of exactly 0.

Now try an overcast, cool, normal-humidity day with weak wind. In the data, it was overcast on 4 days, and tennis was played on all of them. So $P(\text{overcast} \mid \text{no}) = 0/5 = 0$, and the whole “no” product is 0:

$$\text{no: } \frac{5}{14} \times 0 \times \dots = 0$$

The model is 100% sure tennis will be played, purely because one value never happened to appear with “no” in 14 observations. A single zero wipes out all the other evidence, like one veto in a committee that overrules every other vote. With real data and many features, unseen combinations of one value and one class are common, so this is a real problem: training data is never large enough to show every rare event (Manning et al. §13.2).

7. The fix: Laplace smoothing

Key point: Add 1 to every count before dividing. No probability is ever exactly 0, and common values are barely affected.

Laplace smoothing (or **add-one smoothing**) adds a small count, usually 1, to every cell of each crosstab before turning it into probabilities (Manning et al. §13.2):

$$P(\text{value} \mid \text{class}) = \frac{\text{count} + 1}{\text{class count} + \text{number of values}}$$

For outlook given “no”: $(0 + 1)/(5 + 3) = 0.125$ for overcast instead of 0, while sunny becomes $(3 + 1)/8 = 0.5$ instead of 0.6.

Day	Without smoothing	With Laplace smoothing
sunny, hot, high, weak	79.5% no	68.8% no
overcast, cool, normal, weak	100% yes	96.4% yes

The predictions stay the same, but the model is no longer absolutely certain about anything it has seen only a few times.

Python: scikit-learn’s CategoricalNB.

```
from sklearn.naive_bayes import CategoricalNB
from sklearn.preprocessing import OrdinalEncoder
```

```
enc = OrdinalEncoder()
X = enc.fit_transform(df[cols])          # categories -> 0, 1, 2
nb = CategoricalNB(alpha=1.0).fit(X, df["play"])
nb.predict_proba(enc.transform(new_days))
```

CategoricalNB is Naive Bayes for categorical features. Its `alpha` is the smoothing count, 1 by default (scikit-learn docs, CategoricalNB); with a tiny `alpha` it reproduces the unsmoothed numbers above exactly.

8. Summary

- Training: count with `pd.crosstab`, divide by the class sizes, store the lookup table (22 numbers here).
- Testing: multiply the prior by one looked-up probability per feature; the largest score wins.
- Sunny, hot, high, weak gives no (0.0274 against 0.0071).
- A value never seen with a class gives a zero that overrides everything; Laplace smoothing (add 1 to every count) prevents it. `CategoricalNB(alpha=1)` does this by default.

9. Sources

Built from

- CampusX, “Naive Bayes Classifier | Part 8 | Simple Example Code”, YouTube, <https://www.youtube.com/watch?v=DeeWsqaY4Eo>

Other references

- **Manning et al.:** Manning, C. D., Raghavan, P. and Schütze, H. *Introduction to Information Retrieval*. Cambridge University Press, 2008. Section 13.2, “Naive Bayes text classification”.
- **scikit-learn docs:** `sklearn.naive_bayes.CategoricalNB` (alpha), scikit-learn 1.9.

10. Key terms

Term	Meaning
Lookup table	The stored probabilities that Naive Bayes computes during training
<code>pd.crosstab</code>	pandas function that counts how often each pair of values from two columns occurs
Zero-frequency problem	A probability of 0 for a value never seen with a class, which forces that class’s score to 0
Laplace smoothing	Adding a small count (usually 1) to every count so that no probability is 0
<code>CategoricalNB</code>	scikit-learn’s Naive Bayes for categorical features